

中国科学技术大学信息科学技术学院
2023-2024 学年第 2 学期考试试卷

■ A 卷 □ B 卷

课程名称: 计算机原理与嵌入式系统 课程编号: EE2003
开课院系: 信息科学技术学院 考试形式: 半开卷

姓名: 学号: 班级:

题号	一	二	三	四()	四()	四()	四()	总分
得分								

*第四题中选择的题号请填入上表

第二题 (选择题) 答题卡

16	17	18	19	20	21	22	23	24	25
26	27	28	29	30	31	32	33	34	35

第三题 (判断题) 答题卡

36	37	38	39	40	41	42	43	44	45
46	47	48	49	50					

一、填空题 (15 分, 每空 0.5 分, 直接在试卷上答题)

(一) 数据的表示和存取。本课程中实验箱采用 STM32F407 芯片, 存储器配置为小端存储方式, 按字节编址。假设 C 程序中某变量保存在寄存器 R1, R1 内容为 0xFFFF FFF4。综合运用课程知识, 回答如下问题。

1、若该变量是 32 位无符号整数, 变量的真值为 _____, 若该变量经饱和 (Saturation) 操作得到 8 位无符号整数, 该 8 位无符号整数的十进制真值为 _____。
注: 饱和和运算, 即运算结果大于 (数表示法所能表示的) 上限或小于下限时, 结果就等于上限或下限。

2、若该变量是 32 位有符号整数, 变量的十进制真值为 _____, 若该变量经饱和和操作得到 8 位有符号整数, 该 8 位有符号整数的十进制真值为 _____。

3、若 R1 中数值是一个 8 位有符号整数经符号扩展 (Sign Extension) 转换至 32 位得到, 则该 8 位有符号整数的 16 进制真值为 _____, 该 8 位有符号整数的十进制真值为 _____。
注: 有符号整数进行符号扩展时, 扩展的位 (即新增的位) 将复制原始数值的符号位。

4、若 R1 内容保存至地址为 0x2000 0004 ~ 0x2000 0007 的存储单元, 则 0x2000 0004 存放的是 _____, 0x2000 0005 存放的是 _____, 0x2000 0006 存放的是 _____, 0x2000 0007 存放的是 _____。
5、该 STM32F407 系统采用 _____ 级流水线, 其内核为 _____。
6、STM32F407 所用内核的三个关键部件是 CPU、用于中断系统的 _____ 和用于系统定时器的 _____。

(二) Cortex-M3 内核与存储器之间的数据传输。已知某 STM32F103 系统片内仅实现了 32kB 的 SRAM (起始地址 0x2000 0000), 以及 64kB 的 Flash (起始地址 0x0800 0000), 且无外接存储器。系统支持位段操作, 即地址 0x2000 0000 起始的 16B 会被映射至地址 0x2200 0000 起始的别名区域。图 1 为该系统中总线上数据传输的某个片段: 表 1 为某程序执行过程中图 1 操作开始前存储器内存储的部分信息。综合运用课程知识, 并参考“附录 3: STM32F407 系列 MCU 相关信息”和“附录 4: ARM 汇编指令相关信息”信息, 回答下列问题。

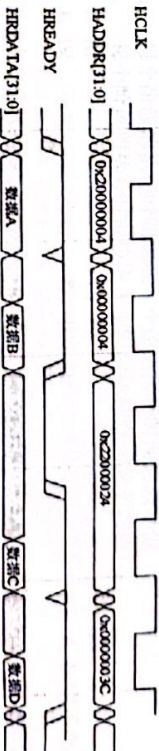


图 1 某 Cortex-M 系统 AHB 总线上的数据传输片段

7、该系统片上 SRAM 可有效寻址的地址范围是 _____。
8、该系统片上 Flash 可有效寻址的地址范围是 _____。
9、图 1 所示时序中数据的传输方向是从 _____ 至 _____。
10、HREADY 信号所起的作用是 _____。

表 1 AHB 总线传输发生前 STM32F103 片内存储器信息

地址	字节数据
0x0000 0000	0x2000 6000
0x0000 0004	0x0800 1000
0x0000 0008	0x0800 2000
...	...
0x0000 003C	0x0800 F0F0
...	...
0x2000 0000	0xFF00 2022
0x2000 0004	0xFFFF 2023
0x2000 0008	0x00FF 2024

11、数据 B 的值是 _____, 存放在 4GB 空间的 _____ 区域。
12、若数据 B 相关的总线操作由指令“LDR R1, [R0]”引起, 则该指令执行前 R0 中保存的数值是 _____。
13、若先将数据 B 内容存放至 R2, 再执行指令“LDR R1, [R2]”, 则指令执行会产生 _____。

存放在 4GB 存储空间的是 ，其含义是 ，该数据是 区域，该区域的主要用途是 。

15、若数据 D 的传输由指令“LDR R1, [R0]”触发，则该指令执行前 R0 中保存的数值是 ，该指令执行后 R1 中保存的数值是 。

二、单项选择题 (30 分，每小题 1.5 分，在试卷首页答题卡答题)

- 16、以下关于 2024H、0010 0001 0010 0001B、2024D 三个数描述正确的是 ()
- 若三个数均为无符号数，则最大的数是 2024H
 - 若三个数均为无符号数，则最大的数是 0010 0001 0010 0001B
 - 若三个数均为无符号数，则最大的数是 2024D
 - 若三个数均为无符号数，则最小的数是 2024H
 - 若三个数均为无符号数，则最小的数是 0010 0001 0010 0001B
 - 若三个数均为无符号数，则最小的数是 2024D
- A) I、V
B) II、VI
C) III、IV
D) III、V
E) IV、V
F) A-E 均不合题意
- 17、某位长 32 的计算机，带符号整数用补码表示，变量 a、b、c、d 的机器数分别为 0x7FFFFFFD、0x00000003、0xFFFFFFFF、0x80000001，下列结论正确的是 ()
- a+b 运算不会出现溢出
 - a+c 运算不会出现溢出
 - b+c 运算不会出现溢出
 - b+d 运算不会出现溢出
 - c+d 运算不会出现溢出
- A) I、II、V
B) I、III、V
C) II、III、IV
D) II、III、V
E) III、IV、V
F) A-E 均不合题意
- 18、下列数值中，不能用 IEEE-754 单精度浮点数精确表示的是 ()
- 1
 - 0.1
 - 1/3
 - 0.25
- A) I、II、V
B) II、III、V
C) II、IV、V
D) II、III、VI
E) III、V、VI
F) A-E 均不合题意
- 19、以下关于微程序控制器中机器指令、微指令、微操作描述正确的是 ()
- 每一条机器指令由一段用微指令编成的微程序来解释执行
 - 每一条机器指令由一条微指令来执行
 - 每一条机器指令的执行可分解为一系列的微操作
 - 每一条微指令由一条机器指令来执行
 - 一段机器指令组成的程序可由一条微指令来执行
 - 每一条微指令由若干条机器指令组成
- A) I、II
B) I、III
C) II、III
D) IV、V
E) IV、VI
F) A-E 均不合题意

20、以下关于计算机体系结构的说法正确的是 ()

- 流水线机制能够有效提高 CPI (Cycles Per Instruction)
 - 流水线机制能够有效提高 IPC (Instructions Per Cycle)
 - Load/Store 体系更有利于实现流水线
 - CISC 相比 RISC 更有利于实现流水线
 - 标量处理器可以使用多条流水线来提升性能
 - 如今智能手机中使用的 Cortex-A 处理器常采用超标量结构
 - 超标量处理器可以使用多条流水线来提升性能
- A) I、II、III
B) II、III、IV
C) III、IV、V
D) IV、V、VI
E) III、VI、VII
F) A-E 均不合题意

21、若将流水线分成 Fetch、Decode、Execute 三个阶段，以下描述正确的是 ()

- 不同指令的不同阶段可以重叠执行
 - 各个阶段都要共用相同的功能部件
 - 各个阶段都要使用相同的总线
 - 每个阶段的执行时间应取 3 个阶段执行时间的最大值
 - 每个阶段的执行时间应取 3 个阶段执行时间的最小值
 - 每个阶段的执行时间应取 3 个阶段执行时间的平均值
- A) I、III
B) I、IV
C) I、V
D) III、IV
E) IV、VI
F) A-E 均不合题意

22、以下指令序列 (指令功能参考“附录 4: ARM 汇编指令相关信息”)，可能导致流水线数据相关的是 ()

- 第 1 条指令为“ADD R0, R1, R2”，第 2 条指令为“ADD R3, R4, R5”
 - 第 1 条指令为“ADD R0, R1, R2”，第 2 条指令为“ADD R2, R1, R0”
 - 第 1 条指令为“ADD R0, R1, R2”，第 2 条指令为“ADD R2, R0, R1”
 - 第 1 条指令为“LDR R1, [R0]”，第 2 条指令为“ADD R0, R1, R2”
 - 第 1 条指令为“LDR R0, [R1]”，第 2 条指令为“ADD R3, R4, R5”
 - 第 1 条指令为“ADD R3, R4, R5”，第 2 条指令为“LDR R1, [R0]”
- A) I、II、III
B) II、III、IV
C) II、III、V
D) III、IV、V
E) IV、V、VI
F) A-E 均不合题意

23、以下关于缓存 (Cache) 地址映射方式说法正确的是 ()

- 全相联映射方式的缓存命中率一般会低于直接相联映射方式
 - 直接相联映射方式的缓存命中率一般会高于组相联映射方式
 - 直接相联映射方式的硬件电路比全相联映射方式要简单
 - 组相联映射方式中，Cache 组内的块数一般取值较小 (如 16)
 - 组相联映射方式中，Cache 的组数一般取值较大 (如 16)
 - 实验所用 STM32F407 系列处理器采用了直接相联映射方式
 - PC 上所用处理器一般会采用组相联映射方式
- A) I、II
B) II、III
C) III、IV
D) IV、VI
E) VI、VII
F) A-E 均不合题意

24、以下关于半导体存储器说法正确的是 ()

- 半导体存储与磁存储、光存储相比，具有更高的可靠性和耐用性
- 固态硬盘 (SSD) 使用了半导体存储技术



- III. U盘使用了半导体存储技术
IV. 硬盘(HDD)使用了半导体存储技术
V. PC的内存条使用了半导体存储技术
VI. 实验所用STM32F407系列芯片使用了半导体存储技术
- A) I, II, III C) II, III, IV
B) I, III, IV D) III, IV, V
- E) III, V, VI
F) A-E均不合题意

25. 以下关于“地址”描述不正确的是()

- I. 地址总线中信号线数目影响了可寻址空间的大小
II. 地址不能用来寻址I/O端口寄存器
III. 地址可以用来寻址存储器内的存储单元
IV. 缓存(Cache)地址空间和主存地址空间是相同的地址空间
V. 紧耦合内存(TCM)地址空间和主存地址空间是各自独立的地址空间
VI. 虚存空间中的虚拟地址可以转换为物理地址
- A) I, II C) III, IV
B) II, III D) IV, V
- E) V, VI
F) A-E均不合题意

26. 以下关于计算机总线说法不正确的是()

- I. 总线的设计思想是把不同部件均连接到一个公共的通道上
II. 发生总线操作意味着CPU内部和外部发生了数据传送
III. 总线上的设备一般分为总线主设备和总线从设备
IV. 连接在同一套总线上的所有设备必须按照统一的时钟工作
V. 无论是单主设备环境还是多主设备环境均需要总线仲裁
VI. 单主设备的总线系统中,总线操作周期可仅包含寻址和数据传输2个阶段
- A) I, II C) III, IV
B) II, III D) IV, V
- E) V, VI
F) A-E均不合题意

27. 以下关于AHB总线描述不正确的是()

- I. AHB总线支持多主设备
II. AHB总线的地址总线复用的
III. AHB的读数据和写数据复用同一组信号线
IV. AHB总线采用了异步总线时序
V. AHB中,“驱动地址”和“驱动数据”可构成两级“流水线”操作
VI. AHB总线的读写通道是分离的
- A) I, II C) II, III
B) I, III D) III, V
- E) IV, VI
F) A-E均不合题意

28. 以下关于AHB突发传输描述不正确的是()

- I. 突发传输可以通过HSIZE[2:0]指定传输数据块的长度
II. 突发传输通过HTRANS[1:0]指示传输状态如“SEQ”、“IDLE”等
III. 突发传输在传输开始前必须指定传输数据的长度
IV. 突发传输所传输数据的地址既可以顺序递增也可以循环递增
V. INCR8类型的突发传输支持的传输数据块长度必须是8个字节
VI. WRAP4类型的突发传输支持的传输数据块长度必须是4个字节
- A) I, II C) III, IV
B) II, III D) IV, V
- E) III, VI
F) A-E均不合题意

29. 某基于Cortex-M4的计算机,存储空间按照字节编址,那么一个对准存放的unsigned int类型临时变量(字数据)的起始地址不会是()

- I. 0x0000 0000 III. 0xFF00 2024
II. 0x2024 2024 IV. 0x2200 2024
A) I, II, III C) III, IV, V
B) II, IV, VI D) III, V, VI
- E) IV, V, VI
F) A-E均不合题意

30. 以下关于实验所用STM32F407系列处理器描述不正确的是()

- I. 复位后默认是非特权访问等级
II. 复位后使用主栈指针MSP
III. 支持AArch64和AArch32两种运行状态
IV. 具有处理模式和线程模式两种操作模式
V. 非特权访问等级可以访问4GB空间的所有区域
VI. 非特权访问等级可以访问所有的寄存器
- A) I, II, III C) I, III, V
B) II, III, IV D) II, IV, V
- E) IV, V, VI
F) A-E均不合题意

31. 以下关于Cortex-M3/M4存储器系统说法不正确的是()

- I. 既可以配置为大端模式,也可以配置为小端模式
II. 地址段0x0000 0000~0x0000 FFFF属于代码(CODE)区
III. 代码区用于存放代码,不能存放数据
IV. 外部RAM区用于存放数据,不能存放代码
V. 内置写缓冲,可以提高程序的执行速度
VI. 存储空间外部RAM区域中有1MB位段被映射为32NB位带别名区域
VII. 处理器读存储器时,存储器系统会检查所访问区域的存储器访问属性
- A) I, II, IV C) III, IV, V
B) II, III, IV D) III, IV, VI
- E) III, V, VII
F) A-E均不合题意

32. 以下关于Cortex-M3/M4堆栈说法不正确的是()

- I. 具有双栈的结构,有MSP和PSP两个堆栈指针
II. MSP和PSP互为影子寄存器,值相同
III. MSP和PSP必须是双字对齐的
IV. 非特权线程模式只能使用PSP而不能使用MSP
V. 在栈内放置一个新的数据后,栈指针数值会增加
VI. 栈空间只能配置在4GB空间的任何位置
VII. 栈空间可以配置在4GB空间的任何位置
- A) I, II, III C) III, IV, V
B) II, III, IV D) IV, V, VI
- E) V, VI, VII
F) A-E均不合题意

33. 以下关于STM32F103系列处理器时钟描述不正确的是()

- I. LSE和LSI时钟源是不能与PLL配合工作的
II. HSE和HSI时钟源由于时钟频率高,需要从芯片外部输入
III. SYSCLK既可以由芯片外部时钟源产生,也可以由芯片内部时钟源产生
IV. 系统定时器SysTick的时钟不能由系统时钟SYSCLK分频得到
V. 独立看门狗模块的时钟可由系统时钟SYSCLK分频得到
VI. 各个GPIO端口的时钟可由系统时钟SYSCLK分频得到
- A) I, II, III B) II, III, IV
C) II, IV, V

D) III、V、VI

E) IV、V、VI

F) A-E 均不合题意

34. 以下关于 STM32 处理器中“地址”描述正确的是 ()

- “IO 端口地址”是和存储器地址统一规划的
 - 特定的“IO 端口地址”代表了特定的寄存器
 - USART1 模块 SR 寄存器的地址不属于“IO 端口地址”
 - GPIOA 模块 ODR 寄存器的地址属于“IO 端口地址”
 - SPI 模块 CR1、CR2 寄存器的地址不属于“IO 端口地址”
 - “PC 总线规范中定义的地址”是 PC 器件内部的存储单元地址
 - “PC 总线规范中定义的地址”是 PC 器件内部的存储单元地址
- A) I、II、III
B) I、II、IV
C) I、III、V
D) III、IV、VI
E) III、V、VI
F) A-E 均不合题意

35. 以下关于 STM32F103 单片机 USART 接口说法不正确的是 ()

- USART 模块既支持异步串行通信, 也支持同步串行通信
 - 触发 USART1 中断的事件不是唯一的
 - USART 中断不能嵌套
 - USART 模块使用前不需要配置引脚复用方式
 - USART 模块可通过对数据寄存器 DR 的读/写实现收发
 - 所有 USART 模块均挂载在 APB1 总线上
- A) I、II
B) II、III
C) III、IV
D) IV、V
E) V、VI
F) A-E 均不合题意

三、判断题 (15 分, 每小题 1 分, 打√或×, 在试卷首页答题卡答题)

- 多级存储体系中引入缓存的作用是解决主存和辅存速度不匹配问题。 ()
- Cortex-M 处理器通过写缓存来提高性能。 ()
- Cortex-M3 系统中, 位带操作可以避免“读-修改-写”问题。 ()
- Cortex-M3 系统中, 中断向量表不能保存在外部 RAM 区和片上外设区。 ()
- Intel 酷睿 i3 代处理器中×8 PCIe 5.0 是典型的并行接口技术。 ()
- Cortex-M3 系统中, 组优先级高的异常可以打断组优先级低的异常。 ()
- Cortex-M3 系统中, 地址线和数据线可以配置为复用方式。 ()
- Cortex-M3 系统中, MSP 的值不能设置为 0x4000 2022。 ()
- AHB 读写数据通道独立, 且总线宽度为 32 位, 故 SRAM 区两个字存储单元交换数据只需要一个总线周期。 ()
- AHB、APB、AXI 均通过支持突发传输来提升传输性能。 ()
- AHB 的流水级 Split 机制允许总线操作的地址阶段和数据阶段分裂开来。 ()
- 一个指令周期可能包含多个总线周期。 ()
- STM32F103 系列 MCU 只有 16 位的定时器, 故而采用 32768 Hz 的 LSI 时, 看门狗的最大延时是 2 秒。 ()
- MCU 把 CPU 和存储器集成在同一芯片中, 故只能采用冯诺依曼结构。 ()
- Cortex-M3 系统中, 异常的优先级寄存器只能在处理模式下访问。 ()

四、设计及综合题 (40 分, 每题 10 分, 5 题选做 4 题, 在稿纸上答题)

(一) CPU 连接存储器

51. 某车载应用系统采用 STM32F103ZE 处理器 (Cortex-M3, 32 根地址线, 32 根数据线), 存储器按字节编址。现有数量充足的车规级 SRAM 芯片, 型号为 Cypress CY7C1011G, 容量为 2-Mbit (规格为 128K × 16-bit)。试为该系统设计一个具有 1MB 容量的片外扩展 SRAM 存储器系统, 要求该 1MB 处于连续地址范围。(提示: Cortex-M3 定义地址范围 0x6000 0000 ~ 0x9FFF FFFF 是外部 RAM 区域)

- [2 分] 计算需要几片 Cypress CY7C1011G。
- [6 分] 画出地址线、数据线及片选信号的连接示意图。
- [2 分] 写出上述设计中扩展 SRAM 存储器系统所占用的地址范围。

(二) Cache 及其应用

52. 某计算机系统主存地址空间大小为 1 MB, 按字节编址。指令 Cache 和数据 Cache 分离, 容量均为 16KB, 均采用 8 路组相联映射方式, 每个 Cache 行 (cache line) 大小为 64B。现有两个功能相同的程序 1 和程序 2, 其伪代码如下所示。

程序 1	程序 2
<pre>int a[64][64]; int sum_array10 { int i, j, sum=0; for(i=0; i<64; i++) for(j=0; j<64; j++) sum += a[i][j]; return sum; }</pre>	<pre>int a[64][64]; int sum_array10 { int i, j, sum=0; for(j=0; j<64; j++) for(i=0; i<64; i++) sum += a[i][j]; return sum; }</pre>

假定 int 类型数据用 32 位补码表示, 程序编译时 i, j, sum 均分配在寄存器中, 数组 a 按行优先方式存放, 其首地址为 20D。请回答下列问题, 要求说明理由或给出计算过程。

- [2 分] 数据 Cache 被分为多少组? 每组内多少个 Cache 行?
- [2 分] 主存被分为多少页? 每页内多少个 Cache 行?
- [2 分] 数组 a 能否全部载入 Cache?
- [2 分] 数组元素 a[0][3] 载入 Cache 时, 写出其对应的 Cache 地址范围。
- [2 分] 计算程序 1 和程序 2 的数据访问命中率 (写出表达式即可)。

(三) GPIO 及定时器

53. 考虑一个 STM32F103 系统, 在其某个 GPIO 端口的引脚连接了 LED 灯 (LEDO) 和拨码开关 (DIP0)。在该系统上执行“附录 1: 定时器控制 LED 亮灭相关代码”所示代码, 能够实现拨码开关接地时, LED 灯按照亮 500 毫秒、灭 500 毫秒规律实现呼吸灯效果。已知该系统配置为使用外部输入的 16 MHz 时钟源且未使能 PLL。回答下列问题。

- [3 分] 根据代码画出最小系统示意图, 要求画出 LED 和 DIP 相关引脚的连接方式。
- [1 分] 写出①行代码中“?”的数值。
- [1 分] 解释②行代码的功能。
- [4 分] 补充完整定时函数的空余部分③④。
- [5 分] 根据“stm32f10x.h”, 分析⑤行代码将访问的存储单元的地址范围 (即 4GB 存储空间内的地址范围), 地址请用 16 进制表示。



(四) GPIO 及中断

54、已知某系统采用 STM32F407 构成最小系统，并在某个 GPIO 端口的引脚连接了一个 LED 灯 (LED0) 和两个拨码开关 (DIP0、DIP1)。在该系统上执行“附录 2：中断控制 LED 的相关代码”所示代码，能够实现拨动开关 DIP0 由不接地（引脚对应位=1）拨至接地（引脚对应位=0）时，触发外部中断，在中断服务子程序中控制 LED 灯的亮灭，回答下列问题。

- [2 分]根据“附录 2：中断控制 LED 的相关代码”中关于 GPIO 寄存器的描述，解释代码行①和②的含义。
- [2 分]补全函数 NVIC_Config0 中③行的代码。
- [3 分]根据附录 2 所示代码，分析拨动开关 DIP0 由不接地至接地时程序的行为。
- [2 分]参考“附录 3：STM32F407 系列 MCU 相关信息”，若主程序中只使用了 DIP0 对应的中断（其他中断未配置），那么在函数 EXTI_Config0 执行后，中断屏蔽寄存器 EXTI_IMR 的值是多少？
- [2 分]参考“附录 3：STM32F407 系列 MCU 相关信息”，修改 GetPinValueFromPA0 函数，使函数功能为“使用位段操作返回 LED0 的状态”。

(五) 串行接口通信

55、图 2 所示为某 Cortex-M3 内核 MCU 与外设的一次字节数据串行传输的过程。SCLK 为同步时钟（时钟频率为 200 kHz），SID 为数据。RW 为数据传送方向控制：H（“1”）表示数据从外设到 MCU，L（“0”）表示数据从 MCU 到外设。在传输 RW 前 SID 需要持续 4 个时钟周期的 H。RS 为数据类型选择：H 表示将要传输的字节为有效数据，L 表示将要传输的字节为控制指令。在传输 RS 后 SID 需要持续 2 个时钟周期的 L。D7-D4 为字节数据的高 4 位，D3-D0 为字节数据的低 4 位。传输过程中，只有在时钟线 SCLK 为低电平时，数据线 SID 上的数据才允许变化。在时钟线 SCLK 为高电平时，SID 上的数据必须保持稳定（不能变化）。

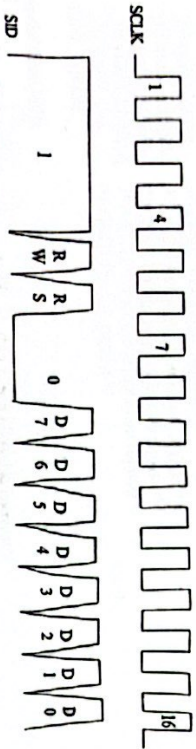


图 2 MCU 与外设的串行数据传输过程示意图

- [1 分]根据图 2 所示数据传输示意图，发送一个字节数据，实际帧数据有多少字节？
- [4 分]假设 MCU 芯片端口寄存器 DEVICE_REG0（32 位）在 4GB 空间的起始地址为 0x4000_2024，该寄存器的 bit0 对应图 2 所示 SCLK 引脚，其 bit1 对应图 2 所示 SID 引脚。请给出对应引脚的位带（Bit-Band）操作的宏定义（即代码“SCLK=1”会在 SCLK 信号线输出“1”，“SID=0”会在 SID 信号线输出“0”）。
- [3 分]假设可调用的延时函数为 delay_us(int N)。根据上述宏定义编写实现数据串行发送一个字节的数据 SendByte。
- [2 分]已知该外设某条指令编码为 0x88，写出 MCU 发该指令操作的代码（调用函数 SendByte 实现）。

附录 1：定时器控制 LED 亮灭相关代码 ST 官方库文件 stm32f10x.h 代码片段

```
#define PERIPH_BASE ((uint32_t)0x40000000)
#define APB1PERIPH_BASE (PERIPH_BASE + 0x10000)
#define APB2PERIPH_BASE (PERIPH_BASE + 0x20000)
#define AHBPERIPH_BASE (PERIPH_BASE + 0x08000)
#define GPIOA_BASE (AHBPERIPH_BASE + 0x00000)
#define GPIOB_BASE (AHBPERIPH_BASE + 0x04000)
#define GPIOC_BASE (AHBPERIPH_BASE + 0x08000)
#define GPIOD_BASE (AHBPERIPH_BASE + 0x0C000)
#define GPIOE_BASE (AHBPERIPH_BASE + 0x10000)
#define GPIOF_BASE (AHBPERIPH_BASE + 0x14000)
#define GPIOG_BASE (AHBPERIPH_BASE + 0x18000)
#define GPIOH_BASE (AHBPERIPH_BASE + 0x1C000)
#define GPIOI_BASE (AHBPERIPH_BASE + 0x20000)
#define GPIOJ_BASE (AHBPERIPH_BASE + 0x24000)
#define GPIOK_BASE (AHBPERIPH_BASE + 0x28000)
#define GPIO_LCKR ((uint32_t)0x00000000)
#define GPIO_ODR ((uint32_t)0x00000000)
#define GPIO_IDR ((uint32_t)0x00000000)
#define GPIO_BSRR ((uint32_t)0x00000000)
#define GPIO_BRR ((uint32_t)0x00000000)
#define GPIO_LCKR ((uint32_t)0x00000000)
#define GPIO_TypeDef
```

主程序代码片段 (main.c)

```
void GPIO_Configuration(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
}

while (1)
{
    if (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_2) == 1) //①
    {
        /*控制 LED 亮灭*/
        GPIO_WriteBit(GPIOA, GPIO_Pin_3, BitSet); //②
        TIM2_Delay500MS();
        GPIO_WriteBit(GPIOA, GPIO_Pin_3, BitReset);
        TIM2_Delay500MS();
    }
}

void TIM2_Delay500MS()
{
    TIM_TimeBaseStructure TIM_TimeBaseStructure;
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
    TIM_TimeBaseStructure.TIM_Prescaler = ③;
    TIM_TimeBaseStructure.TIM_Period = ④;
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_ClearFlag(TIM2, TIM_FLAG_Update);
    TIM_Cmd(TIM2, ENABLE);
    while (TIM_GetFlagStatus(TIM2, TIM_FLAG_Update) == RESET);
}
```



附录 2：中断控制 LED 的相关代码

ST 官方库文件 `stm32f4xx.h` 代码片段

```
#define PERIPH_BASE ((uint32_t)0x40000000)
#define APB1PERIPH_BASE PERIPH_BASE
#define APB2PERIPH_BASE (PERIPH_BASE + 0x00010000)
#define AHB1PERIPH_BASE (PERIPH_BASE + 0x00020000)
#define AHB2PERIPH_BASE (PERIPH_BASE + 0x00030000)
#define GPIOC_BASE (AHB1PERIPH_BASE + 0x10000000)
#define GPIOB_BASE (AHB1PERIPH_BASE + 0x10000000)
#define GPIOA_BASE ((GPIO_TypeDef*) EXTI_BASE)
#define EXTI ((EXTI_TypeDef*) EXTI_BASE)
```

```
typedef struct
{
    __IO uint32_t MODER;
    __IO uint32_t OTYPER;
    __IO uint32_t OSPEEDR;
    __IO uint32_t PUPDR;
    __IO uint32_t IDR;
    __IO uint32_t ODR;
    __IO uint16_t BSRRL;
    __IO uint16_t BSRRH;
    __IO uint32_t LCKR;
    __IO uint32_t IARR[2];
} GPIO_TypeDef;
```

```
typedef struct
{
    __IO uint32_t IMR; /*< EXTI Interrupt mask register */
    __IO uint32_t EMR; /*< EXTI Event mask register */
    __IO uint32_t RTSR; /*< EXTI Rising trigger selection register */
    __IO uint32_t FTSR; /*< EXTI Falling trigger selection register */
    __IO uint32_t SWIER; /*< EXTI Software interrupt event register */
    __IO uint32_t PR; /*< EXTI Pending register */
} EXTI_TypeDef;
```

```
typedef enum IRQn
{
    NonMaskableInt_IRQn = -14, /*< 2 Non Maskable Interrupt */
    MemoryManagement_IRQn = -12, /*< 4 Cortex-M4 Memory Management Interrupt */
    EXTI0_IRQn = 6, /*< EXTI Line0 Interrupt */
    EXTI1_IRQn = 7, /*< EXTI Line1 Interrupt */
    EXTI2_IRQn = 8, /*< EXTI Line2 Interrupt */
    EXTI3_IRQn = 9, /*< EXTI Line3 Interrupt */
    EXTI4_IRQn = 10, /*< EXTI Line4 Interrupt */
    EXTI9_5_IRQn = 23, /*< External Line[9:5] Interrupts */
} IRQn_Type;
```

主程序代码片段 (main.c)

```
//此处略去头文件信息
#define LED0_Pin GPIO_ReadInputDataBit(GPIOC,GPIO_Pin_3)
#define LED0(x) GPIO_WriteBit(GPIOC,GPIO_Pin_3,x) // ①
#define D1Pin GPIO_ReadInputDataBit(GPIOC,GPIO_Pin_4) // ②
#define D1Pin GPIO_ReadInputDataBit(GPIOC,GPIO_Pin_5) //
```

```
void LED0_Config(void);
void KEY0_Config(void);
void EXTI_Config(void);
void NVIC_Config(void);
```

```
int main(void)
{
    LED0_Config(); //
    KEY0_Config(); //
    EXTI_Config(); //
    NVIC_Config(); //
    while(1);
}
```

```
void EXTI_Config(void)
```

```
{
    EXTI_InitTypeDef EXTI_InitStructure;
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG,ENABLE);
    SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOC,GPIO_PinSource4);
    EXTI_InitStructure.EXTI_Line= EXTI_Line4;
    EXTI_InitStructure.EXTI_Mode= EXTI_Mode_Interrupt;
    EXTI_InitStructure.EXTI_Trigger=EXTI_Trigger_Falling;
    EXTI_InitStructure.EXTI_LineCmd= ENABLE;
    EXTI_Init(&EXTI_InitStructure);
}
```

```
void NVIC_Config(void)
```

```
{
    NVIC_InitTypeDef NVIC_InitStructure;
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
    NVIC_InitStructure.NVIC_IRQChannel= ?; // ③
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority=0;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority=1;
    NVIC_InitStructure.NVIC_IRQChannelCmd= ENABLE;
    NVIC_Init(&NVIC_InitStructure);
}
```

```
void LED0_Config(void)
```

```
{
    GPIO_InitTypeDef GPIO_InitStructure;
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC,ENABLE);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_3;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_Init(GPIOC, &GPIO_InitStructure);
}
```

```
void KEY0_Config(void)
```

```
{
    GPIO_InitTypeDef GPIO_InitStructure;
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC,ENABLE);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4|GPIO_Pin_5;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_Init(GPIOC, &GPIO_InitStructure);
}
```



```
void EXTI4_IRQHandler(void)
{
    if (EXTI_GetITStatus(EXTI_Line4) != RESET)
    {
        if (0 == LED0_IsOn())
            GPIO->BSRR = 0x00000008; //①
        else
            GPIO->BSRR = 0x00008000; //②
        EXTI_ClearITPendingBit(EXTI_Line4);
    }
}
```

```
void EXTI9_5_IRQHandler(void)
{
    if (EXTI_GetITStatus(EXTI_Line5) != RESET)
    {
        if (DIP1)
            GPIO->BSRR = 0x00000008;
        else
            GPIO->BSRR = 0x00008000;
        EXTI_ClearITPendingBit(EXTI_Line5);
    }
}
```

```
unsigned int GetPinValueFromPA(void)
{
    //返回物理地址 0x0000000C 起始的 32 位
    return *((volatile unsigned long*)(0x0000000C));
}
```

附录 3：STM32F407 系列 MCU 相关信息

表 2 STM32F407 的 GPIO_BSRR 寄存器位定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0

GPIO_BSRR[15:0] Bx: Port x reset bit y (y=0..15)

Bx: Port x 的输出数据寄存器 (ODRx) 的 y 位置 0, 即 Port x 的输出 0

GPIO_BSRR[15:0] Bx: Port x set bit y (y=0..15)

Bx: Port x 的输出数据寄存器 (ODRx) 的 y 位置 1, 即 Port x 的输出 1

表 3 STM32F407 的 GPIO 输入端口 GPIOx_IDR (x=A..J) 寄存器位定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IN15	IN14	IN13	IN12	IN11	IN10	IN9	IN8	IN7	IN6	IN5	IN4	IN3	IN2	IN1	IN0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

IDR[15:0] IDRx: Port input data (y = 0..15)

表 4 STM32F407 的中断屏蔽 EXTI_IMR 寄存器位定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IM15	IM14	IM13	IM12	IM11	IM10	IM9	IM8	IM7	IM6	IM5	IM4	IM3	IM2	IM1	IM0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

EXTI_IMR[22:0] IMRx: Interrupt mask on line x

IMx 为 0, 表示中断线 x 对应的中断被屏蔽, 系统复位后默认各个位均为 0

IMx 为 1, 表示中断线 x 对应的中断未被屏蔽

表 5 STM32F407 系列 MCU 的中断向量表 (部分)

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	fixed	Reset	Reset	0x0000 0004
-	-	-	-	-	-
-	6	scalable	Systick	System tick timer	0x0000 003C
0	7	scalable	WWDG	Window Watchdog interrupt	0x0000 0040
10	17	scalable	EXTI4	EXTI Line0 interrupt	0x0000 0068
23	30	scalable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000 009C
28	35	scalable	TIM2	TIM2 global interrupt	0x0000 00B0
37	44	scalable	USART1	USART1 global interrupt	0x0000 00D4
38	45	scalable	USART2	USART2 global interrupt	0x0000 00D8
39	46	scalable	USART3	USART3 global interrupt	0x0000 00DC

附录 4：ARM 汇编指令相关信息

ADD 指令

加法指令, 典型语法是 "ADD Rd, Rn, Rm", 该指令将 Rn+Rm 的结果存入 Rd。

示例 1: ADD R1, R2, R3

功能解析: R1 = R2 + R3

示例 2: ADD R1, R1, R2

功能解析: R1 = R1 + R2

LDR 指令

从存储器读取数据至寄存器。典型语法为 "LDR Rd, [Rn, #offset]", 该指令从存储器位置 Rn+offset 读取一个字存入寄存器 Rd, 若指定操作数地址方式为变址寻址 (从位置 Rn+offset 读取一个字, 更新 Rn 为 Rn+offset), 其典型语法为 "LDR Rd, [Rn, #offset]!"。若指定操作数地址方式为后变址寻址 (从位置 Rn 读取一个字, 更新 Rn 为 Rn+offset), 其典型语法为 "LDR Rd, [Rn!], #offset"。

示例 1: LDR R3, [R4]

功能解析: R4 中保存的是一个字数据的地址, 读取该地址起始的 4 个字节至 R3 寄存器。

示例 2: LDR R3, [R4, #0x8]

功能解析: 操作数 (字数据) 的地址是 R4 中数值加 0x8, 读取该操作数 (字数据) 至 R3 寄存器。

示例 3: LDR R3, [R4, #0x8]!

功能解析: 先将 R4 中数值加 0x8, 再把 R4 中数值作为操作数 (字数据) 的地址, 读取该操作数 (字数据) 至 R3 寄存器。